

Click-to-Source 3D

Stage 4 Dogfooding Report

Applying the Tool to a Real Procedural Project

1. Executive Summary

Stage 4 was defined as: apply the Stage 3 tool to a real project — the Procedural Low-Poly World Generator — and use it to fix three known regressions (normals ordering, weather freeze, lake bed height clamp).

What actually happened was different in kind, not just in detail. The tool could not be applied as-is. Attempting to tag the first real generator exposed a gap in the AST editor. Attempting to tag the second exposed that the engine had no support for `InstancedMesh` at all — which covers most objects in the scene. Attempting to tag the fourth exposed a silent failure mode in which every editable field appeared correct and failed only on save.

Stage 4 therefore became two stages running at once: dogfooding, and the engine work that dogfooding proved was missing. Four unplanned capability changes shipped inside a stage scoped as validation. That is a strong outcome — the gaps were real, they were found by real use rather than speculation, and each is now closed and tested — but it is not the outcome the plan described, and this report records the difference rather than smoothing it over.

Final position: two of three regressions resolved, one deliberately descope, four objects tagged and verified in a running browser, four engine capabilities added, 13 commits across two repositories, and the test suite grown from 25 to 38.

2. Original Scope and Final Status

Regression	Status
Lake bed height clamp	Resolved — surfaced through the tool; drift bug found and fixed
Normals ordering	Resolved — fixed via geometry change, with numerical proof
Weather freeze	Out of scope — deliberately dropped after two rebuild attempts

The weather decision is the significant one and is documented in full in Section 6. It was not left unresolved by accident; it was removed from scope after the cost of repair repeatedly exceeded its value to the stage's actual objective.

3. Integration

Click-to-Source was linked into the Procedural Low-Poly World Generator as a local `file:` dependency rather than published to npm, keeping the two repositories separate while allowing live iteration on the engine.

Integration required:

- `@click-to-source/core` and `@click-to-source/overlay` linked via `file:` paths
- The dev-only Vite file-I/O plugin (`/__cts/read-file`, `/__cts/write-file`) registered in the consuming project's `vite.config.js`
- `SelectionHighlight` mounted inside the R3F Canvas, `GenerationTrace` mounted outside it
- A `<group onPointerUp>` wrapper around scene meshes, calling `useClickToSource()` and writing results into the overlay store

A Three.js version conflict surfaced during this step: the workspace carried `three@0.170.0` in core and `three@0.185.1` in overlay and examples, producing multiple physical installations and Three's own multiple-instance warning. A specific claim — that this was already causing `instanceof` failures — was checked against the source and **rejected**: the resolver walks `userData` and `parent` structurally and performs no constructor identity checks anywhere. The duplication was still corrected on general grounds, normalising the workspace to `0.170.0`, later confirmed compatible with `@react-three/fiber@9.7.0`'s declared peer range of `>=0.156`.

4. Object 1 — Terrain

Terrain was the first real generator tagged, and it immediately produced two findings.

4.1 The `VariableDeclarator` gap

The lake bed clamp values were hoisted into local constants:

```
const noiseFloor = -30, lakeBedLevel = -20;
```

Editing either through the panel returned `ARGUMENT_NOT_FOUND`. Investigation of `sourceEditor.ts` found that `collectCandidates` only recognised two AST node types: `JSXAttribute` and `ObjectProperty`. Hoisted variable declarations — the way real generators actually write shared values — were invisible to the editor.

Two fixes were possible: rewrite the generator to fit the parser, or extend the parser to fit real code. The second was chosen deliberately. Rewriting source to satisfy tooling would have established an unwritten rule that the tool dictates how consumers must write their code, which is the wrong direction for a tool whose purpose is to observe code as written.

`VariableDeclarator` support was added to the core editor, along with `UnaryExpression` handling so negative numeric literals resolve correctly, plus three tests covering literal replacement, line-based disambiguation of duplicate names, and the ambiguous-location error path.

4.2 The metadata drift bug

The `SourceRef` initially hardcoded its args:

```
args: { noiseFloor: -26, lakeBedLevel: -20 }
```

while the code had since moved to `-30`. The panel confidently displayed a value that no longer existed in the source. This is the first instance of a pattern that recurred throughout Stage 4: hand-authored provenance drifts silently from the code it describes.

The fix was structural rather than corrective — the constants were hoisted to component scope and the args changed to shorthand:

```
args: { noiseFloor, lakeBedLevel }
```

so the displayed value is the same value the geometry uses, by construction. Correcting the number alone would have fixed the symptom and left the mechanism intact.

Note on attribution. The lake bed clamp itself predates the tool — implemented in `b939b0c` and refined in `ca34a46`, both before the Click-to-Source integration in `1e8b2a1`. What the tool did was surface the clamp's parameters for inspection and editing, and catch the drift bug in the process. Stage 4 did not fix this regression; it made it visible and verifiable.

5. Object 2 — Trees, and the InstancedMesh Problem

Tagging trees exposed the largest gap in the stage.

5.1 The structural problem

Stage 3's entire resolution model assumed one rendered object maps to one `Object3D` carrying one `userData.sourceRef`. `THREE.InstancedMesh` breaks that assumption completely: 260 trees are rendered across five instanced meshes (one trunk group, four foliage colour groups), each sharing a single `Object3D`, a single `userData`, and a single material. Clicking any tree resolved to whatever the parent walk found — never to the specific instance.

Compounding this, an instance's generating values are not literals. Position, scale, and yaw come from a seeded RNG inside a placement loop. There is no line of source corresponding to "the tree at x = -135.678".

5.2 The design, in three phases

Phase 1 — core resolution. A new `InstanceSourceRef` type in `@click-to-source/shared` wrapping a standard `SourceRef`. Generators store an array of these on the mesh as `userData.instanceSourceRefs`, indexed 1:1 by Three.js `instanceId`. `resolveSourceRef()` gained an optional `instanceId` parameter; a module-private `resolveInstanceSourceRef()` performs the lookup, returning `null` — never throwing — on undefined `instanceId`, missing array, out-of-bounds index, or malformed entry. Every `null` falls through to the Stage 3 parent walk unchanged, which is the backward-compatibility guarantee.

Phase 2 — read-only presentation. Per-instance provenance is read-only by construction: `readonly: true` is hardcoded in the resolver, because an instance's args are RNG outputs with no editable literal behind them. Offering an input for them would promise a write-back that cannot exist. This required extending the overlay store to carry `instanceId` and `readonly` — an audit found it was silently dropping both fields before the UI ever

saw them — and updating `GenerationTrace` to render plain text instead of inputs when the flag is set.

Phase 3 — producer wiring. `Trees.jsx` builds `instanceSourceRefs` arrays alongside its existing matrix arrays, pushing into both within the same loop iteration so indices cannot desync. A single shared `instanceRef` object is pushed into both the trunk and foliage arrays for a given tree, so clicking either resolves to identical provenance — one tree, one record.

5.3 Three bugs found by actually clicking

Automated verification passed at every step. The tool remained unusable until three further problems were found by hand:

Trees were unclickable — raycasts fell through to Terrain. `InstancedMesh` raycasting requires explicit bounding volumes. Without `computeBoundingBox()` and `computeBoundingSphere()` called after the instance matrices are written, the raycast tests against the base geometry's bounds — a single tree at the origin — and misses everything. The diagnosis was confirmed by a controlled comparison: `GroundCover.jsx` already made both calls, which is precisely why grass was clickable before any of this work while trees were not.

`instanceId` was never forwarded. The resolver was extended to accept it and `resolveObjectAtPoint` was updated to extract it from the intersection — but `useClickToSource`, the hook that actually receives R3F's click event, still called `resolveSourceRef(event.object)` with one argument. Instance resolution could never fire.

`SelectionHighlight` was commented out. An unrelated leftover from an earlier debugging session, correctly identified as a separate accidental regression rather than part of the instancing work. Restoring it also required adding an `OutputPass` to the composer chain, without which the entire scene rendered dark — `EffectComposer` renders in linear colour space and needs the final conversion back to sRGB.

6. Weather — Descoped

Weather consumed more of Stage 4 than any other single item and produced no shipped fix. The full history matters, because the decision to drop it was made against evidence rather than fatigue.

First diagnosis overturned the assumed cause. The freeze had been attributed to synchronous FBM noise computation. Investigation found this was wrong — the cloud and ice FBM ran correctly in GLSL fragment shaders, and snow terrain sampling already ran in a Web Worker. The real cost was in `SnowfallSystem`: 350 particles per frame, each performing a full `Object3D.updateMatrix()`, with the same camera-facing Euler copied into every particle, triggering 350 identical Euler-to-quaternion conversions per frame on a two-core machine.

A second diagnosis found the symptoms were entangled. The camera appeared frozen, the day/time slider appeared to move the camera, and weather defaulted to Snow rather than Clear. The default alone confounded every test — the camera had never been evaluated without the heaviest weather state running.

Weather was removed, then rebuilt, then removed again. The v2 rebuild addressed every documented cause: `InstancedMesh` particles instead of individual `Object3Ds`, billboard rotation computed once per frame, reduced particle budgets, conditional mounting so Clear costs nothing. It built clean and passed static checks. It also broke immediately on selecting any non-Clear preset — a mount-transition bug, in code where browser testing had never actually been performed because the automation tooling failed both times it was attempted.

The decision. Weather is not one of Click-to-Source's dogfooding targets in any meaningful sense — it is a performance and lifecycle problem in the consuming project that happens to share a repository. Two attempts had consumed multiple sessions with no shipped result. Weather was moved out of Stage 4 scope entirely and recorded as a possible future feature with no scheduled stage. [docs/weather-v2-notes.md](#) is preserved as the record of what was learned across all three attempts.

This is recorded as a scope decision, not an unresolved bug.

7. Objects 3 and 4 — GroundCover and Water

7.1 GroundCover

Grass and bushes across five instanced meshes (three grass variants, two bush variants). The pattern established for Trees applied directly, with one deliberate design difference: grass and bushes receive fully independent `instanceRef` objects rather than sharing one, because unlike a tree's trunk and foliage they are not two views of a single entity.

This was the smoothest tagging pass in the stage — no new engine work required — which was the first real evidence that the InstancedMesh capability generalised beyond the case that motivated it.

7.2 Water, and the naming mismatch

Water is a single mesh with genuinely editable literals, so unlike the instanced objects it should support write-back. It did not. All four fields failed.

The root cause took two attempts to find. The first hypothesis — that the constants needed to share one declaration line so a single `sourceRef.line` could address them — was correct but insufficient. After merging them, all four still failed.

The actual cause: `editSource` matches its `argName` against the identifier **as declared in source**. Nothing had ever required `sourceRef.args` keys to equal those identifiers, and the coupling was undocumented. Terrain satisfied it by accident — `args: { noiseFloor, lakeBedLevel }` uses shorthand, so keys and declarations are identical. Water used camelCase display keys against SCREAMING_CASE module constants, and every edit failed.

The fix added an optional `argSources` map from display key to declared identifier, resolved in `editSourceFile` before the request is built. Keys absent from the map resolve to themselves, so existing consumers are unaffected. Resolution was deliberately placed in the overlay rather than core: translating a `SourceRef` into an `EditRequest` is already `editSourceFile`'s responsibility, and core's contract stays narrow — *given a file, a line, and a declared identifier, rewrite the literal*.

A fifth field, `terrainSize`, was removed rather than fixed. It is a prop with no literal in the file; it traces to `ALPINE_TERRAIN.worldSize`, which Terrain, Trees, and GroundCover all share. Editing it from the Water panel would have silently resized the entire world. Shipping a field that always fails, or one that lies about its blast radius, were both worse than omitting it.

8. The Normals Ordering Fix

The last regression, and the one where the tool's role was diagnostic rather than corrective.

The bug. `Terrain.jsx` called `toNonIndexed()` before `computeVertexNormals()`. Converting to non-indexed gives every triangle its own vertices, so computing normals afterwards produces per-face normals with no smoothing anywhere.

Why it mattered more than "terrain looks faceted." In a low-poly generator, faceting is often intentional, so the symptom alone was not proof. The shader settles it. `Terrain.jsx:146` blends two normals:

```
normal = normalize(mix(normal, flatNrm, flatFactor));
```

with an explicit comment documenting the intent: smooth normals on grass, flat normals on mountains and cliffs. Because the attribute normals were already flat, the smooth end of that blend was unreachable — the mix was a no-op in one direction. A `mix()` with a dead branch is a regression, not a style choice.

Numerical proof rather than visual judgement. The real terrain geometry was rebuilt in Node with the same generator, seed, and segment count, and normals computed both ways:

	Current order	Proposed order
Triangles with identical vertex normals (flat)	115,200 / 115,200 (100%)	16 / 115,200 (0.0%)
Triangles with varying vertex normals (smooth)	0	115,184

Zero smooth normals existed in the buffer. The 16 remaining flat triangles under the fix are expected — the `LakeBedLevel` clamp creates genuinely coplanar regions.

Two fixes evaluated, and the less obvious one chosen. Option A was to reorder the two calls. Option B was to drop `toNonIndexed()` entirely and keep the geometry indexed. Option B won, because the conversion turned out to be vestigial rather than merely mis-ordered: the shader derives its flat normal from screen-space derivatives ($dFdx/dFdy$) and seeds its per-face colour hash from the same derivatives. Neither reads the normal attribute or depends on topology. The conversion was buying nothing the shader was not already computing itself.

Both options were verified to produce identical shader inputs — max delta `0.000e+0` across all 115,200 triangles on geometric face normals, the `faceID` hash, `terrainMasks` per-triangle values, and vertex normals. Option B additionally reduced the geometry from 345,600 to 58,081 vertices, roughly 14.50 MB to 3.10 MB, and cut vertex shader invocations by about six times — material on the two-core, 2 GB-VRAM target hardware this project is built against.

`GroundCover.jsx` contains superficially similar calls and was deliberately left untouched: its `toNonIndexed()` flattens a blade template for sequential reading, and its `computeVertexNormals()` runs on a different geometry built without an index. There is no order to reverse there, and a naive "fix both call sites" would have broken grass merging.

9. Verification

Automated coverage grew alongside the engine work: 25 tests at the start of Stage 4, 38 at the end, across five files. All builds green at close — `tsc -b` clean, full suite passing, consuming project building.

Manual browser verification was performed by hand, because automated browser tooling failed on every attempt across the entire stage. Verification depth differs per value and is recorded honestly rather than rounded up:

Value	Depth of verification
Terrain, Trees, GroundCover, Water — resolution	Confirmed in a running browser; distinct per-instance values confirmed across instances
<code>Water.waterLevel</code> — full edit loop	Confirmed in browser: panel edit, disk write, HMR reload, correct value on reselection
<code>Water.deepColor</code> , <code>Water.shallowColor</code>	Direct <code>editSourceFile</code> calls only — not confirmed in browser
<code>Terrain.noiseFloor</code> , <code>Terrain.lakeBedLevel</code>	Direct <code>editSource</code> calls only — not confirmed in browser
Trees, GroundCover instance args	Read-only by construction; no edit path exists to verify

10. Process Notes

Two aspects of how Stage 4 was executed are worth recording, because both affected the outcome.

Automated verification consistently failed to catch the bugs that mattered. Every significant defect in this stage — the drift bug, the missing bounding volumes, the unforwarded `instanceId`, the commented-out highlight, the naming mismatch — passed builds and tests and was found by a person clicking something. The consuming project has no test suite at all, which means the entire real-world feedback loop for this tool is manual. That is recorded as an open gap, not resolved.

A cross-repository commit accident required a recovery. Work intended for a separate repository was committed into Click-to-Source, and the subsequent rollback used a mixed reset and force-push that left the entire Stage 3 implementation uncommitted and briefly absent from the remote. Nothing was lost — the files remained on disk and were recommitted after a full audit — but the incident established the working discipline used for the rest of the stage: stage files by exact name, never `git add .`, review every diff before commit, and verify claims with real command output rather than assertion.

11. Commits

click-to-source (9 pushed at close): `argSources` display-key resolution and its tests; `"type": "module"` corrections across all three packages; per-instance resolution test coverage; the Stage 5 InstancedMesh addendum; the Stage 4 exit condition; and three gap records.

Earlier in the stage: [2c444f6](#) (VariableDeclarator support and OutputPass fix), [eff2c21](#) (read-only overlay support), [5d03a07](#) (Phase 1 instance resolution).

Procedural Low-Poly World Generator (4 pushed at close): GroundCover per-instance provenance; Water tagging with [argSources](#); the terrain normals fix; and the weather scope decision.

Earlier in the stage: [1e8b2a1](#) (integration and weather removal), [0542070](#) (Terrain args derived from real variables), [1fe8012](#) (SelectionHighlight restored), [63604ba](#) (bounding volumes and line number correction).

Push order mattered: [9eca538](#) in the consuming project depends on [282a12f](#) upstream, so click-to-source was pushed first to avoid a window where the consuming project referenced an [argSources](#) field that did not exist in the published package.

12. Open Items

Recorded rather than quietly dropped:

- Four values never verified through the browser panel (Section 9)
- Mesh-wide selection highlight for instanced objects — clicking one tree outlines every tree sharing that mesh, because [OutlinePass](#) receives the whole [InstancedMesh](#). Data resolution is correctly per-instance; only the visual is mesh-wide. [instanceId](#) is already in the overlay store, so a fix is a consumption change rather than new plumbing.
- Read-only validation at panel open — an arg with no resolvable literal still renders as an editable input and fails only on save. [argSources](#) makes the mismatch expressible; it does not make it detectable.
- No test for the [resolveObjectAtPoint](#) raycast-to-[instanceId](#) integration path — precisely the path that failed with the bounding-volume bug.
- No test suite in the consuming project.

13. Triage Rule for Future Stages

Stage 4 absorbed four unplanned capability changes because no rule existed for deciding what belonged in the stage. Going forward:

In-stage — a gap is fixed within the current stage only if it blocks that stage's stated objective. The [InstancedMesh](#) work qualified: most of the scene is instanced, so per-instance provenance could not be demonstrated at all without it.

Deferred — a gap that is real but does not block the stated objective is recorded in the relevant architecture document's Known Gaps section and scheduled, not fixed in-stage.

Every deferred gap must be written down when found. A stage closes when its stated objectives are met and its deferrals are recorded — not when no known gaps remain.

14. Assessment

Stage 4 did not go according to plan, and the ways it diverged are the most useful thing it produced.

The plan assumed a finished tool applied to known bugs. What happened instead was that real use immediately exposed the tool as unfinished in three specific, unanticipated ways — an AST parser too narrow for how people actually write code, no support for the rendering pattern most of the scene uses, and an undocumented coupling between display keys and source identifiers that failed silently. None of these were predictable from the Stage 3 demo scene, because that scene was built to demonstrate the mechanism rather than to stress it.

That is the strongest argument for dogfooding as a practice, and it is the honest headline for this stage: the tool now works on a real project because a real project proved it did not.

The recurring failure mode is worth naming as a single pattern rather than five separate bugs. Hand-authored provenance drifts, mismatches, and mislocates — the args said `-26` while the code said `-30`; the line said 176 while the block was at 178; the display key said `waterLevel` while the declaration said `WATER_LEVEL`. Each was fixed individually. The mechanism that produces them was not, and cannot be while tagging is manual. That is the central input Stage 4 provides to the automatic-instrumentation decision the project has deferred since Stage 2.

Stage 4 is closed.